

Proyecto Lenguajes de Programación

Título del Proyecto:

ESPOL Quest: Videojuego de plataformas 2D para la exploración y orientación en el campus Gustavo Galindo

Integrantes del Grupo:

- Kevin Gálvez
- José Gaviño
- Jorge del Campo

1. Problemática a resolver

Los estudiantes de nuevo ingreso (novatos) de la Escuela Superior Politécnica del Litoral (ESPOL) se enfrentan sistemáticamente a una severa desorientación espacial e institucional al incorporarse al campus Gustavo Galindo. La extensión territorial del campus, junto con la dispersión geográfica de sus facultades, laboratorios, comedores, áreas de atención de Bienestar Estudiantil y la Biblioteca Central, dificulta la pronta adaptación del alumnado. La falta de familiaridad con la distribución física y con las normativas académicas básicas genera pérdidas considerables de tiempo, impuntualidad y episodios de estrés durante las primeras semanas lectivas.

Los mecanismos de inducción tradicionales (folletos impresos, charlas masivas o mapas estáticos en formato PDF) resultan poco interactivos e ineficaces para captar el interés de la población estudiantil actual. En consecuencia, subsiste una curva de aprendizaje y adaptación lenta que afecta la integración inicial del estudiante a la vida universitaria. Se requiere una solución tecnológica dinámica y atractiva que transforme el proceso de reconocimiento del campus en una experiencia formativa e interactiva.

2. Objetivos Propuestos

Objetivo General:

- Desarrollar un videojuego educativo de plataformas 2D que enseña a estudiantes de nuevo ingreso la distribución física, facultades y servicios del campus ESPOL mediante la exploración de niveles temáticos, resolución de misiones y mecánicas de gamificación.

Objetivos Específicos:

- **Kevin Alejandro Gálvez Gavilanes:** Diseñar e implementar el motor y escenario 2D de plataformas que representa las zonas y facultades del campus Gustavo Galindo, permitiendo el desplazamiento, interacción con puntos de interés y mecánicas de física básica.
- **José Gaviño:** Diseñar e implementar el motor dinámico de misiones y trivias integradas en los niveles, que evalúe normativas y ubicaciones con retroalimentación inmediata al jugador.
- **Jorge del Campo:** Desarrollar el sistema de gestión de perfiles de usuario, seguimiento de progreso (niveles superados, insignias y porcentaje de exploración) y la tabla de clasificación/ranking global.

3. Alcance del Proyecto

Describe las limitaciones y el alcance del proyecto. Esto puede incluir qué se cubrirá y qué no se cubrirá, así como cualquier restricción técnica o de recursos.

Qué cubre el proyecto:

- Reconstrucción en estilo plataformas 2D (side-scrolling / vista lateral o cenital) de los sectores clave del campus Gustavo Galindo (FIEC, FCNM, FIMCP, FCSH, Biblioteca Central, Bienestar Estudiantil, etc.).
- Control del personaje (movimiento, salto, interacción con checkpoints/edificios).
- Sistema de misiones tipo *quiz/trivia* gatilladas al alcanzar checkpoints o interactuar con elementos dentro del nivel.
- Guardado de progreso, desbloqueo de niveles/insignias y tabla de posiciones.

Qué NO cubre el proyecto:

- No incluye gráficos ni física 3D.
- No incluye georreferenciación GPS en tiempo real.
- Cobertura limitada al Campus Prosperina (excluye Peñas o Santa Elena).

4. Características de la Solución

Mundos y Niveles de Plataformas 2D: Escenarios interactivos donde el jugador controla un avatar para recorrer las facultades y servicios del campus.

Puntos de Interés / Checkpoints Interactivos: Objetos o puertas en el nivel que, al ser tocados, despliegan fichas informativas y trivias académicas.

Mantenimiento del resto de módulos: Sistema de misiones/trivias, Sistema de puntaje y ranking, Perfil de progreso del jugador y Panel de administración (CMS).

5. Requerimientos Funcionales Asignados

Requerimiento	Descripción	Responsable
Gestionar escenario y físicas 2D	Controla el renderizado de niveles, movimiento del avatar, colisiones con plataformas y activación de puntos de interés.	Kevin Gálvez
Consultar selector de niveles / mapa de juego	Muestra el mapa general de mundos/niveles disponibles para seleccionar qué zona del campus explorar.	Kevin Gálvez
Registrar respuestas de misiones en nivel	Captura las respuestas del jugador al interactuar con un checkpoint dentro del nivel de plataformas, valida y otorga puntaje.	José Gaviño
Consultar misiones y trivias disponibles	Muestra el catálogo de misiones asignadas a cada nivel/facultad con sus respectivas preguntas y estado.	José Gaviño
Registrar progreso e insignias del jugador	Guarda el porcentaje de niveles superados, insignias obtenidas y nivel actual del jugador.	Jorge del Campo
Consultar tabla de clasificación (ranking)	Muestra la tabla de líderes global y filtrada por facultad según el puntaje obtenido en los niveles.	Jorge del Campo

7. Arquitectura de la Aplicación

Tipo de Aplicación:

Aplicación Web Progresiva (PWA) con renderizado en Canvas HTML5. Permite acceso directo desde navegadores móviles sin requerir tiendas de aplicaciones.

Patrón de Arquitectura:

Modelo-Vista-Controlador (MVC) desacoplado mediante una arquitectura basada en microservicios distribuidos que se comunican vía API REST:

- Servicio Principal de Juego (Python - Flask): Encargado del bucle del juego, gestión de estados de nivel, colisiones, procesamiento de respuestas de misiones en tiempo real y consumo de endpoints.
- Servicio de Administración y Ranking (Ruby - Ruby on Rails): Encargado de la persistencia de perfiles, sistema de logros/insignias, CMS para administración de trivias/preguntas y motor del ranking global y por facultad.

Estrategia de Comunicación:

Ambos servicios backend operan de forma independiente y se comunican a través de peticiones HTTP/HTTPS en JSON, compartiendo la persistencia de datos mediante una base de datos PostgreSQL 15.

Herramientas y Frameworks:

- Motor Gráfico 2D / Frontend: Phaser.js (o PixiJS) integrando React.js para interfaces administrativas/UI.
- Backend 1: Python 3.11 con Flask framework y SQLAlchemy ORM.
- Backend 2: Ruby 3.2 con Ruby on Rails (API mode) y Active Record.
- Base de Datos: PostgreSQL 15.

8. Lenguajes de Programación

Back-end:

- **Lenguaje 1: Python (Framework Flask)**
 - **Pros:** Sintaxis altamente legible y limpia que facilita el desarrollo ágil; ecosistema extenso de bibliotecas; microframework ligero ideal para manejar la lógica central del juego con mínima sobrecarga computacional; rápida integración con ORMs.
 - **Contras:** Tipado dinámico que exige disciplina de pruebas; menor velocidad bruta de ejecución frente a lenguajes compilados; limitaciones en multithreading por el GIL (Global Interpreter Lock).

- **Lenguaje 2: Ruby (Framework Ruby on Rails)**
 - **Pros:** Principio de "Convención sobre Configuración" que agiliza la creación de componentes CRUD; excelente madurez del ecosistema de gemas; desarrollo estructurado y mantenible, perfecto para administrar perfiles, trivias y tablas de clasificación.
 - **Contras:** Mayor consumo de memoria RAM; curva de aprendizaje más pronunciada en las convenciones estrictas del framework; rendimiento inferior en procesamiento numérico intensivo.
- Evaluación según Criterios de la Asignatura:
 - **Tipado:** Ambos lenguajes presentan tipado dinámico y fuerte. Python ofrece Type Hints opcionales que mejoran el análisis estático en el IDE.
 - **Rendimiento:** Flask ofrece un menor footprint de memoria y un inicio de servicio más rápido para microservicios pequeños; Rails optimiza los tiempos de desarrollo para operaciones I/O complejas con la base de datos.
 - **Curva de Aprendizaje:** Python posee una curva de aprendizaje sumamente suave. Ruby on Rails requiere asimilar la convención DSL del framework.
 - **Comunidad y Madurez del Ecosistema:** Ambos entornos cuentan con comunidades globales masivas, soporte a largo plazo y paquetes de producción altamente consolidados.

Front-end:

- **Lenguaje 1: JavaScript (Librería React.js + Motor 2D Phaser.js)**
 - **Pros:** Lenguaje nativo del navegador, sin pasos de compilación obligatorios; ecosistema NPM masivo con integración directa tanto para las interfaces en React.js como para el canvas del juego con Phaser; modelo asíncrono (promesas, async/await) ideal para el consumo de las APIs REST de Flask y Rails.
 - **Contras:** Tipado dinámico y débil con coerción implícita que permite errores de tipo detectables solo en tiempo de ejecución; dificulta el refactoring de lógicas complejas de física y colisiones a medida que crece el juego; requiere herramientas externas (ESLint) para verificación estática.
- **Lenguaje 2: TypeScript (React + Phaser con TSX)**
 - **Pros:** Tipado estático y estructural que detecta errores en tiempo de compilación; permite usar interfaces y genéricos para documentar estrictamente los estados del jugador, coordenadas del mapa 2D y contratos de las APIs REST consumidas; autocompletado y refactoring superiores en el IDE.
 - **Contras:** Exige un paso adicional de transpilación y configuración del toolchain; curva de aprendizaje extra por el sistema de tipos; puede ralentizar el prototipado rápido inicial en un proyecto académico corto.

- **Evaluación según Criterios de la Asignatura:**
 - **Tipado:** JavaScript posee tipado dinámico y débil; TypeScript añade tipado estático y estructural verificado en tiempo de compilación, aportando robustez a la programación orientada a objetos que requieren los personajes y escenarios.
 - **Rendimiento:** Equivalente en ejecución, ya que TypeScript se transpila a JavaScript y el navegador ejecuta JS sobre el Canvas en ambos casos.
 - **Curva de Aprendizaje:** JavaScript es más accesible para iniciar; TypeScript exige aprender su sistema de tipos, interfaces y genéricos.
 - **Comunidad y Madurez del Ecosistema:** Ambos cuentan con ecosistemas masivos sobre NPM; TypeScript es el estándar de facto en proyectos de mediana/gran escala y cuenta con soporte oficial riguroso para motores de juegos web.

9. Prototipo de Baja Fidelidad

Incluya capturas de pantalla de un prototipo de baja fidelidad utilizando una herramienta de diseño/interacción. Puede utilizar herramientas como MarvelApp, Proto.io, Figma, entre otras.

Herramienta de Diseño Utilizada: Figma

Capturas: Pantallas principales del prototipo: inicio y registro del jugador, mapa interactivo del campus, misión con trivia, perfil de progreso y tabla de clasificación.

10. Implementación

11.Backend

10. Referencias

Incluya cualquier referencia o fuente que haya utilizado para la elaboración de esta propuesta en formato IEEE o APA. No olvide las citas dentro del documento.

Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining “gamification”. En Proceedings of the 15th International Academic MindTrek Conference (pp. 9–15). ACM. <https://doi.org/10.1145/2181037.2181040>

Agafonkin, V. (s.f.). Leaflet: An open-source JavaScript library for mobile-friendly interactive maps. <https://leafletjs.com>

Pallets Projects. (s.f.). Flask documentation. <https://flask.palletsprojects.com>

Ruby on Rails. (s.f.). Ruby on Rails guides. <https://guides.rubyonrails.org>

Meta Open Source. (s.f.). React. <https://react.dev>

Microsoft. (s.f.). TypeScript documentation. <https://www.typescriptlang.org/docs>

